

# Produtor–consumidor com pipes e semáforos

Fala guiada e referência técnica para apresentação individual de 15 minutos.

OBJETIVO DA FALA

Mostrar como a mesma ideia — produzir, transmitir e consumir itens — muda quando usamos **dois processos com pipe** ou **várias threads com um buffer compartilhado**.

## Distribuição do tempo

| Tempo       | O que fazer                                                               |
|-------------|---------------------------------------------------------------------------|
| 0:00–1:00   | Introduzir o problema e distinguir processos de threads.                  |
| 1:00–6:00   | Atividade 2: explicar pipe/fork, demonstrar dez números, mostrar término. |
| 6:00–12:30  | Atividade 3: explicar buffer e três semáforos, demonstrar 30 itens.       |
| 12:30–14:00 | Explicar como ler os gráficos e o que depende das medições.               |
| 14:00–15:00 | Comparar soluções e responder perguntas.                                  |

## Prepare antes de começar

- Use Linux/WSL, com **g++** e **make**. Deixe o terminal aberto na raiz do repositório e os dois READMEs à mão.
- Amplie a fonte. Mostre no máximo três pontos de cada arquivo; evite percorrer o código inteiro.
- Use entradas pequenas ao vivo. O benchmark completo tem 280 execuções e não cabe na apresentação.

ABERTURA — DIGA EM VOZ ALTA

“Vou mostrar duas versões do produtor–consumidor. Na primeira, pai e filho são processos separados e trocam números por um pipe. Na segunda, várias threads compartilham um buffer e precisam de semáforos para coordenar o acesso.”

# 1. Contexto necessário

**Produtor–consumidor** é um padrão: uma parte gera itens, outra os processa. Se trabalham em velocidades diferentes, é preciso guardar itens temporariamente e definir quando cada lado deve esperar.

| Conceito       | Entenda assim                                                                                     | Neste trabalho                                          |
|----------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Processo       | Programa em execução, com PID e espaço de memória próprio.                                        | Pai e filho da atividade 2.                             |
| Thread         | Linha de execução dentro de um processo; compartilha dados com outras threads desse processo.     | Produtoras e consumidoras da atividade 3.               |
| Bloqueio       | Execução fica esperando um evento, sem girar continuamente na CPU.                                | Leitura do pipe; espera por vaga ou item nos semáforos. |
| Região crítica | Trecho que altera dados compartilhados e não pode ser executado simultaneamente por duas threads. | Alteração do vetor, índices e contadores.               |

## DISTINÇÃO PRINCIPAL

No **pipe**, o problema é levar dados entre processos separados e definir o fim do fluxo. Com **threads**, os dados já estão no mesmo processo; o problema é impedir corridas e controlar buffer cheio ou vazio.

## Condição de corrida, em 20 segundos

Se duas produtoras leem um contador com valor 5, ambas podem calcular 6 e gravar 6. Duas operações ocorreram, mas o contador avançou uma vez só. O resultado passou a depender da ordem de execução. O semáforo de exclusão mútua protege essa alteração.

## Uma ressalva se perguntarem “onde está o distribuído?”

Esta demonstração roda em uma máquina. Ela exercita mecanismos básicos de comunicação e coordenação que ajudam a compreender sistemas distribuídos; não demonstra comunicação entre computadores diferentes.

## FRASE DE TRANSIÇÃO

“Primeiro vamos usar dois processos: como eles não compartilham diretamente as variáveis do programa, criaremos um canal para os números.”

## 2. Atividade 2 — pipe entre pai e filho

**Tempo sugerido:** 5 minutos. Abra `atividade-2-pipes/README.md` e depois `src/producer_consumer.cpp`.

### 2.1 Antes de executar — diga

#### FALA

“O programa cria o pipe antes de chamar `fork()`. Assim, pai e filho recebem acesso ao mesmo canal. O pai fecha a ponta de leitura e produz; o filho fecha a ponta de escrita, lê e testa primalidade. O fluxo é pai → pipe → filho.”

### 2.2 Demonstração — digite

```
cd atividade-2-pipes
make
./bin/producer_consumer.out 10
```

**Observe:** devem aparecer dez classificações do tipo “Consumidor: [valor] é/não é primo”. Os valores não são fixos porque os incrementos são sorteados; a sequência deve crescer.

### 2.3 Enquanto a saída aparece — diga

#### FALA

“O produtor parte de **N0 = 1** e soma um incremento aleatório entre **1 e 100** a cada novo valor. O consumidor recebe cada valor e verifica se ele é primo. A entrada 10 define quantos números serão produzidos.”

### 2.4 Encerramento — diga

#### FALA

“Após enviar os dez números, o produtor envia **0**. Esse zero é uma sentinela de término, não um número a classificar. O pai fecha a escrita e chama `waitpid()` para aguardar o filho.”

#### SE A PROFESSORA PERGUNTAR

O pipe é um **fluxo de bytes**. O programa definiu mensagens de **20 bytes** para delimitar os valores. As rotinas de leitura e escrita continuam até completar a transferência, mesmo que uma chamada faça apenas parte dela.

### 3. O que mostrar no código do pipe

Arquivo: **atividade-2-pipes/src/producer\_consumer.cpp**. Mantenha a explicação nesta ordem; os números de linha são referências aproximadas do commit atual.

| Trecho         | O que apontar             | O que explicar                                                            |
|----------------|---------------------------|---------------------------------------------------------------------------|
| Linhas 195–218 | pipe(), fork() e close(). | Canal criado antes do filho; cada processo fecha a ponta que não utiliza. |
| Linha 113      | run_producer().           | Incrementos aleatórios; escrita das mensagens; zero ao final.             |
| Linha 150      | run_consumer().           | Leitura, decodificação, parada no zero e teste de primalidade.            |
| Linha 221      | waitpid().                | O pai aguarda a conclusão do filho.                                       |

#### Perguntas que podem surgir

**Por que criar o pipe antes do fork?** Para que pai e filho herdem as pontas do mesmo canal.

**Por que fechar as pontas não utilizadas?** Cada processo conserva apenas a direção de que precisa. Isso também permite tratar o fechamento do fluxo corretamente.

**Por que 20 bytes?** É o protocolo escolhido para que os dois lados saibam o tamanho de cada valor enviado.

**Por que zero?** Os valores gerados são positivos; zero pode sinalizar o fim sem se confundir com um item válido.

**O pipe substitui a memória compartilhada?** Nesta solução, sim: pai e filho passam os números pelo canal, em vez de usar um vetor compartilhado para os itens.

#### TRANSIÇÃO PRONTA

“No pipe, resolvemos a troca de dados entre processos. Agora as produtoras e consumidoras serão threads do mesmo processo. Elas poderão acessar o mesmo vetor, então precisamos controlar **quem entra e quando existe vaga ou item.**”

## 4. Atividade 3 — buffer e semáforos

**Tempo sugerido:** 6 a 7 minutos. Abra `atividade-3-semaforos/README.md` e `src/producer_consumer_semaphores.cpp`.

### 4.1 Antes de executar — diga

#### FALA

“Agora temos **N<sub>p</sub>** threads produtoras, **N<sub>c</sub>** consumidoras e um vetor circular de **N** posições. Todas compartilham índices e contadores. Sem sincronização, duas threads poderiam alterar esses dados ao mesmo tempo e causar uma condição de corrida.”

| Semáforo     | Valor inicial | Papel                                                               |
|--------------|---------------|---------------------------------------------------------------------|
| mutex        | 1             | Exclusão mútua na alteração do vetor, índices e contadores.         |
| empty_slots  | N             | Conta vagas livres; produtora espera se o buffer está cheio.        |
| filled_slots | 0             | Conta itens disponíveis; consumidora espera se o buffer está vazio. |

### 4.2 Demonstração — digite

Você terminou a atividade 2 dentro da pasta `atividade-2-pipes`. Continue com:

```
cd ../atividade-3-semaforos
make
./bin/producer_consumer_semaphores.out 10 2 2 30 ocupacao_demo.csv
head ocupacao_demo.csv
```

**Explique antes de pressionar Enter:** 10 vagas no buffer, 2 produtoras, 2 consumidoras e 30 itens ao todo. O CSV opcional registra a ocupação após cada produção ou consumo.

#### O QUE OBSERVAR

Ao final, procure **Itens produzidos/consumidos: 30/30** e o tempo em milissegundos. O CSV começa com **operacao,ocupacao**. A ocupação deve ficar entre 0 e 10 e terminar em 0. Valores e ordem das linhas podem variar.

## 5. Explique a sincronização passo a passo

Arquivo: `atividade-3-semaforos/src/producer_consumer_semaphores.cpp`. Mostre a classe **SharedBuffer** (linha 33), **producer** (linha 142) e **consumer** (linha 173).

| Produtora                                         | Consumidora                                       |
|---------------------------------------------------|---------------------------------------------------|
| 1. Espera uma vaga em <code>empty_slots</code> .  | 1. Espera um item em <code>filled_slots</code> .  |
| 2. Obtém mutex e insere o valor.                  | 2. Obtém mutex e retira o valor.                  |
| 3. Atualiza índice e contadores; libera mutex.    | 3. Atualiza índice e contadores; libera mutex.    |
| 4. Anuncia um item em <code>filled_slots</code> . | 4. Anuncia uma vaga em <code>empty_slots</code> . |

### FALA PRINCIPAL

“**mutex** impede alterações simultâneas na região crítica. Os outros dois semáforos controlam a disponibilidade: vagas para produzir e itens para consumir. A consumidora calcula a primalidade **depois** de retirar o item e liberar o acesso ao buffer; assim o cálculo não segura a região crítica.”

### Buffer circular

Quando um índice chega à última posição do vetor, retorna à primeira. Por exemplo, com três posições:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ . Isso permite reutilizar as vagas liberadas sem deslocar todos os itens.

### Como o programa termina

O padrão é **M = 100.000** itens; a demonstração passa **M = 30** para ser rápida. Depois que as produtoras terminam, a thread principal marca o fim da produção e acorda consumidoras eventualmente bloqueadas. Elas podem sair quando não houver mais itens. O programa confere que produzidos e consumidos são iguais a M e que o buffer ficou vazio.

### DISTINÇÃO IMPORTANTE

O pipe transmite um **zero sentinela** pelo canal. A solução com threads encerra por **estado compartilhado e sinalização**, sem inserir itens artificiais no buffer.

## 6. Como apresentar os gráficos

O script `scripts/benchmark.sh` prevê **4 capacidades × 7 combinações de threads × 10 repetições = 280 medições**. Ele registra um histórico de ocupação representativo para cada uma das 28 configurações.

### Gráfico de tempo médio: `tempo_medio.svg`

#### FALA

“No eixo horizontal estão combinações como **1P/1C** e **4P/1C**. No eixo vertical está o **tempo médio em milissegundos**; menor significa mais rápido. Cada curva corresponde a uma capacidade N. Cada ponto é a média de dez execuções.”

Para analisar, escolha **dois pontos reais** e compare seus valores. Mais threads ou mais espaço no buffer não garantem ganho: pode haver disputa pela CPU e pelo acesso à região crítica. Não atribua uma causa única sem dados adicionais.

### Gráficos de ocupação: `ocupacao_*.svg`

#### FALA

“O eixo horizontal mostra a **ordem das operações** de produção e consumo, não segundos. O vertical mostra quantas posições estão ocupadas, entre 0 e N. A linha sobe quando alguém produz e desce quando alguém consome. Uma ocupação frequentemente próxima de N sugere produtoras esperando vagas; próxima de zero sugere consumidoras esperando itens.”

O histórico é da **primeira repetição** de cada cenário; ele ilustra a dinâmica do buffer. A linha sozinha não prova qual configuração foi mais rápida. Para isso, consulte as médias de tempo.

#### LIMITE DOS DADOS DESTA CÓPIA

Nesta pasta há apenas um CSV local de demonstração; **timings.csv** e os SVGs do benchmark completo não estavam presentes durante a preparação deste roteiro. Não diga que uma configuração venceu ou cite tempos sem abrir resultados reais. Se você tiver os gráficos em outro computador, compare os valores mostrados neles.

## 7. Perguntas prováveis e plano B

| Pergunta                                   | Resposta curta                                                                                                                             |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Por que mutex sozinho não basta?           | Ele impede acesso simultâneo, mas não informa se há vaga ou item. Para isso existem <code>empty_slots</code> e <code>filled_slots</code> . |
| Por que o pipe não precisa de mutex?       | A troca de itens ocorre por um canal entre processos; nesta solução, cada ponta tem um papel definido.                                     |
| O que é uma condição de corrida?           | Resultado incorreto ou variável quando a ordem de operações concorrentes altera dados compartilhados.                                      |
| Por que a saída das threads muda de ordem? | O escalonador decide quando cada thread executa. Na atividade 2, o consumidor mantém a ordem dos números recebidos pelo pipe.              |
| Por que não rodar o benchmark ao vivo?     | São 280 execuções. A demonstração pequena mostra o mecanismo; os resultados completos devem ser medidos antes e analisados separadamente.  |

### Se a demonstração falhar

- Se faltar **g++** ou **make**, mostre o README, a chamada do programa e o fluxo no código. Diga qual saída seria esperada; não invente que a execução ocorreu.
- Se a saída mudar, explique o sorteio dos números e o escalonamento. Compare as propriedades, não os valores exatos.
- Se o tempo estiver acabando, corte a leitura do código. Preserve: problema → mecanismo → uma execução → encerramento.

#### FECHAMENTO — DIGA EM VOZ ALTA

“As duas versões resolvem produtor–consumidor de maneiras diferentes. Entre processos, o pipe carrega os números e uma sentinela marca o fim. Entre threads, o buffer já é compartilhado; os semáforos protegem o acesso e controlam vagas e itens. Essa comparação mostra quando precisamos de comunicação e quando precisamos de sincronização.”

Base desta referência: READMEs, código-fonte, script de benchmark e relatório local do repositório `sd-tp1`, commit `48fb2f1`. O projeto não foi executado para produzir este PDF.